

UNIVERSIDADE FEDERAL DE UBERLÂNDIA

Laura Rosado Rodrigues Muniz

**Algoritmo Genético aplicado ao problema do
8-puzzle**

Uberlândia, Brasil

2025

UNIVERSIDADE FEDERAL DE UBERLÂNDIA

Laura Rosado Rodrigues Muniz

Algoritmo Genético aplicado ao problema do 8-puzzle

Trabalho de conclusão de curso apresentado à Faculdade de Computação da Universidade Federal de Uberlândia, como parte dos requisitos exigidos para a obtenção título de Bacharel em Sistemas de Informação.

Orientador: Profa. Dra. Christiane Regina Soares Brasil

Universidade Federal de Uberlândia – UFU

Faculdade de Computação

Bacharelado em Sistemas de Informação

Uberlândia, Brasil

2025

Laura Rosado Rodrigues Muniz

Algoritmo Genético aplicado ao problema do 8-puzzle

Trabalho de conclusão de curso apresentado à Faculdade de Computação da Universidade Federal de Uberlândia, como parte dos requisitos exigidos para a obtenção título de Bacharel em Sistemas de Informação.

Trabalho aprovado. Uberlândia, Brasil, 16 de setembro de 2025:

Profa. Dra. Christiane Regina Soares
Brasil
Orientadora

Prof. Dr. Bruno Augusto Nassif
Travencolo

Prof. Dr. Paulo Henrique Ribeiro
Gabriel

Uberlândia, Brasil
2025

Resumo

Este trabalho apresenta a implementação e análise de um Algoritmo Genético (AG) para a solução do problema do quebra-cabeça de 8 peças, visando investigar a eficiência desse método em problemas de busca combinatória. O objetivo principal foi desenvolver uma abordagem que combinasse operadores de seleção, *crossover* e mutação para encontrar a solução ótima ou próximas da ótima em diferentes configurações do quebra-cabeça. O método consistiu na representação do estado do quebra-cabeça como cromossomos, a aplicação de operadores genéticos e a avaliação da população com base em uma função de aptidão. Os experimentos demonstraram que o AG é capaz de encontrar soluções viáveis com número limitado de movimentos e apresentou desempenho consistente em diferentes taxas de mutação, *crossover* e elitismo, evidenciando a importância do equilíbrio entre exploração e diversidade da população. Os resultados obtidos mostraram-se bastante satisfatórios quando comparados à literatura, confirmando a relevância do ajuste adequado dos parâmetros. Conclui-se que a aplicação de AGs em problemas de busca, como o 8-puzzle, proporciona resultados promissores, sendo possível ajustar parâmetros para otimizar eficiência e qualidade das soluções.

Palavras-chave: Algoritmo Genético. 8-puzzle. Otimização. Problemas de busca. Inteligência Artificial.

Abstract

This study presents the implementation and analysis of a Genetic Algorithm (GA) applied to the solution of the 8-puzzle problem, with the purpose of investigating the efficiency of this method in combinatorial search problems. The main objective was to develop an approach that combined selection, crossover, and mutation operators to identify optimal or near-optimal solutions under different puzzle configurations. The method consisted of representing puzzle states as chromosomes, applying genetic operators, and evaluating the population based on a fitness function. The experiments demonstrated that the GA could find viable solutions within a limited number of moves and showed consistent performance across different mutation, crossover, and elitism rates, highlighting the importance of balancing exploration and population diversity. The results obtained were highly satisfactory when compared to the literature, confirming the relevance of proper parameter tuning. It is concluded that the application of GAs to search problems, such as the 8-puzzle, yields promising results, making it possible to adjust parameters to optimize both efficiency and solution quality.

Keywords: Genetic Algorithm. 8-puzzle. Optimization. Search problems. Artificial Intelligence.

Lista de ilustrações

Figura 1 – Diagrama de fluxo de um AG.	15
Figura 2 – Representação do operador de mutação padrão.	17
Figura 3 – Recombinação 1-ponto e 2-pontos.	18
Figura 4 – Recombinação uniforme com $P = 0.5$	18
Figura 5 – Método da Roleta.	20
Figura 6 – Método de Torneio.	20
Figura 7 – Quebra-cabeça deslizante de 8 peças.	21
Figura 8 – Representação do funcionamento da função de <i>crossover</i>	27
Figura 9 – Função de mutação - modificação de movimento no cromossomo atual.	28
Figura 10 – Função de mutação - adição de movimento no cromossomo atual.	28

Lista de tabelas

Tabela 1 – Taxa de mutação = 1%.	33
Tabela 2 – Taxa de mutação = 3%.	34
Tabela 3 – Taxa de mutação = 5%.	35
Tabela 4 – Taxa de mutação = 15%.	35
Tabela 5 – Taxa de mutação = 30%.	36
Tabela 6 – Resumo comparativo entre as taxas de mutação.	37
Tabela 7 – Taxa de elitismo = 1%.	38
Tabela 8 – Taxa de elitismo = 5%.	39
Tabela 9 – Taxa de elitismo = 15%.	39
Tabela 10 – Taxa de elitismo = 20%.	40
Tabela 11 – Resumo comparativo entre as taxas de elitismo,	41
Tabela 12 – Taxa de <i>crossover</i> = 100%.	42
Tabela 13 – Taxa de <i>crossover</i> = 95%.	43
Tabela 14 – Taxa de <i>crossover</i> = 90%.	43
Tabela 15 – Taxa de <i>crossover</i> = 80%.	44
Tabela 16 – Resumo comparativo entre as taxas de <i>crossover</i>	45
Tabela 17 – Comparação entre os AGs do trabalho de referência e do trabalho desenvolvido.	45
Tabela 18 – Trabalho de referência - Taxa de mutação.	47
Tabela 19 – Experimento com o AG desenvolvido - Taxa de mutação.	47

Lista de abreviaturas e siglas

AE	Algoritmo Evolutivo
AG	Algoritmo Genético
CE	Computação Evolutiva
IA	Inteligência Artificial

Sumário

1	INTRODUÇÃO	10
1.1	Objetivo	11
1.2	Justificativa	11
1.3	Estrutura do texto	11
2	FUNDAMENTAÇÃO TEÓRICA	13
2.1	Algoritmo Evolutivo	13
2.1.1	Algoritmo Genético	13
2.1.2	Base Biológica	16
2.1.3	Operadores de Reprodução	16
2.1.3.1	Mutação	16
2.1.3.2	Recombinação	17
2.1.4	Métodos de Seleção	19
2.2	Teoria dos Jogos e o Problema do Quebra-Cabeça de 8 peças	20
2.3	Trabalhos Relacionados	22
2.4	Considerações Finais	23
3	IMPLEMENTAÇÃO DO ALGORITMO GENÉTICO PARA O PROBLEMA 8-PUZZLE	25
3.1	Desenvolvimento do Algoritmo Genético	25
3.2	Classe Pad: Representação do Tabuleiro	25
3.3	Classe Chromosome: Representação do Indivíduo	27
3.4	Classe Solver: Solução do quebra-cabeça	29
3.4.1	Inicialização da População	29
3.4.2	Cálculo de aptidão (<i>Fitness</i>)	29
3.4.3	Seleção e elitismo	30
3.4.4	Recombinação	30
3.4.5	Mutação	31
3.4.6	Solucionador	31
3.5	Considerações Finais	32
4	RESULTADOS	33
4.1	Experimentos com a taxa de mutação	33
4.1.1	Análise dos resultados	36
4.2	Experimentos com a taxa de elitismo	38
4.2.1	Análise dos resultados	40

4.3	Experimentos com a taxa de <i>crossover</i>	42
4.3.1	Análise dos resultados	44
4.4	Comparação com trabalho relacionado	45
4.4.1	Análise dos resultados	47
4.5	Considerações Finais	48
5	CONCLUSÃO	50
	REFERÊNCIAS	51

1 Introdução

A Computação Evolutiva (CE) é uma área de pesquisa que se baseia em conceitos biológicos para desenvolver algoritmos geralmente aplicados na busca por soluções de problemas complexos. Esta área tem se expandido rapidamente, principalmente pela capacidade de criar algoritmos eficientes que encontram soluções de qualidade para estes problemas, com certa simplicidade em sua implementação por serem métodos de otimização que utilizam uma lógica intuitiva inspirada no processo evolutivo na natureza. Dentre eles, tem-se os Algoritmos Evolutivos (AE) ([GABRIEL; DELBEM, 2008](#)).

Os AEs têm como base a Teoria da Evolução das Espécies, descrita por [Darwin \(1859\)](#), e utilizam princípios como seleção natural, reprodução e mutação de indivíduos para a otimização de problemas. Neste sentido, destacam-se os Algoritmos Genéticos (AG), um tipo de AE, propostos por John Holland em 1975 em seu trabalho intitulado de “*Adaptation in Natural and Artificial Systems*” ([HOLLAND, 1975](#)), que considerou a evolução natural como um processo robusto e simples que poderia ser adaptado utilizando os operadores de reprodução para a obtenção de soluções computacionais eficientes para problemas de otimização ([GABRIEL; DELBEM, 2008](#)). Desta forma, os Algoritmos Evolutivos (AEs) abrangem um conjunto de técnicas inspiradas na evolução natural, enquanto os Algoritmos Genéticos (AG) representam uma abordagem específica dentro deste grupo, distinguindo-se pelo uso de estruturas semelhantes a cromossomos e pela aplicação dos operadores de seleção, cruzamento e mutação obrigatoriamente, enquanto os AEs não necessariamente precisam aplicar o cruzamento para que haja evolução, por exemplo, podendo evoluir a população utilizando apenas mutação ([EIBEN; SMITH, 2003](#)).

A aplicação de AEs aos mais diversos problemas permite a exploração de espaços de solução (complexos e não lineares) ao lidar com um ou mais objetivos e encontrar soluções adequadas para os mesmos. No entanto, estes algoritmos são sensíveis aos parâmetros, exigindo ajustes para alcançar soluções ótimas e garantir um desempenho adequado. A performance dos AEs também é influenciado pela escolha dos operadores genéticos, métodos de seleção e pela forma como o problema é representado. Embora existam várias abordagens de AEs, o princípio básico é o mesmo: dada uma população de indivíduos, ou conjunto de soluções, ocorre um processo de seleção natural, em que os indivíduos mais aptos são privilegiados e consistirão uma nova população. O processo é repetido até que um número máximo de iterações (gerações) seja alcançado ou até que uma solução suficientemente qualificada seja apresentada.

Neste trabalho, o Algoritmo Evolutivo, especificamente o Algoritmo Genético, será aplicado ao problema do quebra-cabeça deslizante de 8 peças, também conhecido como

8-puzzle. Esse problema tem como objetivo reorganizar uma configuração inicial das 8 peças quadradas de um tabuleiro de dimensão 3×3 , em uma configuração final previamente definida, ao mover, sucessivamente, as peças ortogonalmente adjacentes a uma peça vazia presente no tabuleiro (SHABAN; ALKALLAK; SULAIMAN, 2010; YANG et al., 2023).

1.1 Objetivo

O objetivo geral deste Trabalho de Conclusão de Curso foi implementar o Algoritmo Genético aplicado ao problema do quebra-cabeça de 8 peças, visando realizar um estudo a partir da análise dos experimentos e resultados obtidos.

O objetivo específico foi realizar, a partir dos experimentos, uma análise comparativa dos parâmetros utilizados e dos operadores genéticos do algoritmo implementado neste trabalho com a investigação realizada por Yang (YANG et al., 2023).

1.2 Justificativa

O uso de AGs para solucionar problemas de busca e otimização tem crescido devido à sua eficiência em encontrar soluções de qualidade (ótimas ou próximas de ótimas) em problemas de alta complexidade, onde o custo computacional cresce à medida que a dimensão do problema aumenta. Deste modo, o estudo da aplicação do algoritmo genético ao problema do quebra-cabeça de 8 peças contribui para a aplicação de técnicas de Inteligência Artificial (IA), mais especificamente da área da Computação Evolutiva, no campo dos jogos e da otimização.

Os resultados deste estudo podem ser aplicados não apenas no desenvolvimento de algoritmos mais eficientes para o 8-puzzle, mas também em outras áreas que envolvem a resolução de problemas complexos de busca, uma vez que o 8-puzzle modela situações reais em que um conjunto de elementos deve ser reorganizado em um espaço restrito com regras de movimentação específicas — como em robótica de armazéns, automação de estoques, ou sistemas embarcados com reconfiguração de módulos. Além disto, o trabalho permitirá confirmar a relevância do ajuste de parâmetros em algoritmos genéticos, evidenciando como esse processo impacta diretamente nos resultados obtidos.

1.3 Estrutura do texto

Este trabalho foi organizado da seguinte maneira:

- Capítulo 1: apresenta a contextualização do problema, os objetivos e a justificativa da pesquisa.

-
- Capítulo 2: expõe os conceitos essenciais sobre Algoritmos Genéticos e revisa estudos relacionados ao tema.
 - Capítulo 3: descreve a implementação do Algoritmo Genético e as ferramentas utilizadas.
 - Capítulo 4: apresenta os testes realizados e os resultados obtidos.
 - Capítulo 5: relata as conclusões finais e sugestões para trabalhos futuros.

2 Fundamentação Teórica

Neste Capítulo, serão apresentados os principais conceitos sobre Algoritmos Evolutivos e o problema do quebra-cabeça de 8 peças, abordando definições essenciais relacionadas ao tema. Além disto, também neste Capítulo 2 serão mencionadas algumas pesquisas relacionadas ao tema deste trabalho.

2.1 Algoritmo Evolutivo

Os Algoritmos Evolutivos (AEs) são técnicas de otimização inseridas na área de Computação Bioinspirada, que se fundamentam em diversos conceitos biológicos, especialmente naqueles derivados da Teoria da Evolução de Charles Darwin (DARWIN, 1859)(GABRIEL; DELBEM, 2008).

Neste contexto, a utilização dos AEs tem se expandido cada vez mais, uma vez que são capazes de encontrar soluções ótimas ou aproximações das mesmas para problemas complexos, utilizando métodos relativamente simples de desenvolvimento e podendo ser adaptados para as mais diversas áreas do conhecimento (GABRIEL; DELBEM, 2008).

Na literatura, podem ser encontradas três abordagens de algoritmos evolutivos: a Programação Evolutiva (PE), que prioriza mutações e seleção baseada no desempenho das soluções; as Estratégias Evolutivas (EEs), que adaptam parâmetros de busca junto com as soluções, sendo comuns em problemas de otimização contínua; e os Algoritmos Genéticos (AGs), que utilizam seleção, cruzamento e mutação inspirados na genética biológica (GABRIEL; DELBEM, 2008).

Na próxima seção, serão tratados os principais conceitos de Algoritmo Genético.

2.1.1 Algoritmo Genético

Os Algoritmos Genéticos (AGs) foram propostos por Holland e seus alunos na década de 1970 (HOLLAND, 1975; GOLDBERG, 1989), que acreditavam que a técnica fundamentada na teoria evolucionista traria soluções computacionais eficientes para problemas de otimização (GABRIEL; DELBEM, 2008).

De modo geral, os AGs possuem como diferencial das outras abordagens a criação de descendentes obrigatoriamente pelo operador de recombinação (JONG, 2006). Além disto, pelo fato de utilizar de operadores genéticos, como cruzamento (recombinação) com a possibilidade de mutação, o algoritmo genético consegue colocar em equilíbrio dois objetivos conflitantes: o aproveitamento das melhores soluções e a exploração do espaço de

busca. Desta forma, o processo de busca preserva soluções candidatas e promove a troca de informação entre os indivíduos explorados (MICHALEWICZ, 1996; ZUBEN, 2000).

O princípio básico do funcionamento de um AE é: dado uma população inicial de indivíduos, ou conjunto de soluções, haverá uma seleção natural em que os mais aptos poderão ser privilegiados (dependendo do tipo de seleção aplicada). Inicialmente, um conjunto de soluções é gerado aleatoriamente dentro do domínio da função objetivo e é calculado o *fitness* de cada solução, a fim de medir sua adequação ao problema. Indivíduos com alta aptidão poderão ser selecionados para gerar novas soluções por meio de recombinação e/ou mutação, que serão inseridas na população atual. Baseando-se em seu *fitness*, são selecionados indivíduos que serão mantidos para a próxima geração, de acordo com uma taxa do elitismo pré-estabelecida. Este processo é repetido, criando gerações sucessivas, até que uma solução suficientemente boa seja encontrada ou um limite de iterações seja alcançado (GABRIEL; DELBEM, 2008).

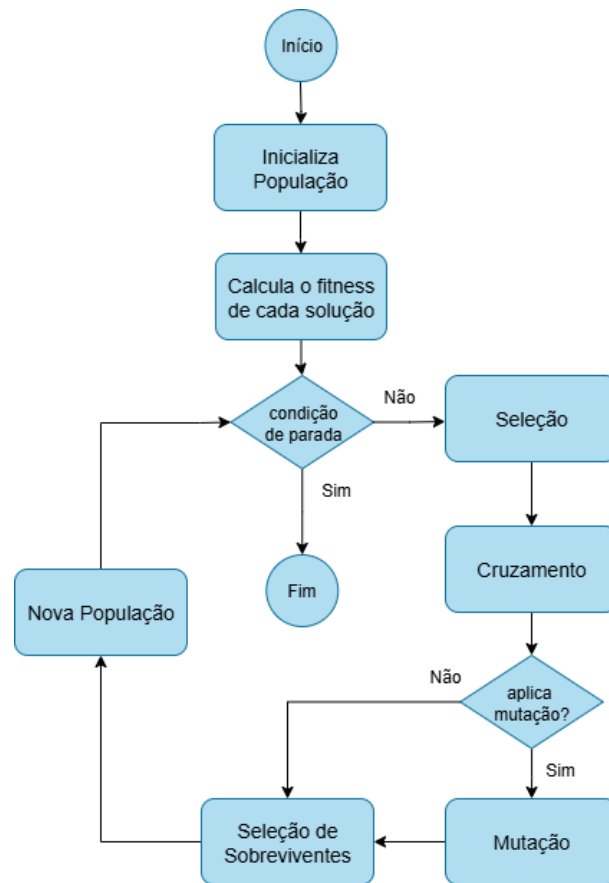
Os passos de um Algoritmo Genético são descritos na pseudocódigo (Algoritmo 1), apresentado a seguir.

Algoritmo 1 Pseudocódigo de um Algoritmo Evolutivo Típico.

- 1: **GERAR** população inicial aleatoriamente
 - 2: **AVALIAR** cada solução
 - 3: **repita**
 - 4: **SELECIONA** indivíduos para reprodução
 - 5: **REALIZA** cruzamento entre os selecionados
 - 6: **REALIZA** mutação nos descendentes, caso necessário
 - 7: **AVALIA** *fitness* das novas soluções
 - 8: **SELECIONA** soluções para a próxima geração
 - 9: **até** CONDIÇÃO DE PARADA satisfeita
-

A Figura 1 apresenta o fluxograma de um AG típico.

Figura 1 – Diagrama de fluxo de um AG.



Nos Algoritmos Genéticos, as soluções candidatas podem ser representadas em vetores em codificação binária, inteira, real ou em ponto flutuante. Segundo (MICHALEWICZ, 1996) e (DEB, 2001), a representação binária pode levar a um desempenho não satisfatório para problemas em que é requerida alta precisão numérica. Portanto, AGs com representação inteira ou em ponto flutuante costumam apresentar um desempenho melhor em relação à codificação binária.

Como fora explicado, os Algoritmos Genéticos utilizam métodos de seleção de indivíduos tanto para escolher os indivíduos pais (métodos da roleta ou seleção por torneio) quanto para manter alguns deles para a próxima geração. Entre eles, estão os métodos de *ranking* e seleção elitista (GABRIEL; DELBEM, 2008). O desempenho do algoritmo pode ser melhorado ao utilizar a seleção elitista que, de modo geral, mantém uma taxa dos melhores indivíduos da geração atual nas gerações seguintes. Os métodos de seleção serão mais bem definidos na Seção 2.1.4.

A seguir, serão explicados mais detalhadamente os fundamentos advindos da Biologia.

2.1.2 Base Biológica

A forma como os indivíduos são construídos é definida por estruturas denominadas cromossomos. Estas estruturas contêm o material genético (DNA) de um organismo, organizado em forma de longas cadeias (AMABIS; MARTHO, 1985). O DNA é o responsável por armazenar informações codificadas para uma determinada característica de um indivíduo (AMABIS; MARTHO, 1985). A saber, os valores de um gene são denominados alelos, e a posição de um gene em um cromossomo é denominada locus (COELLO; VELDHUIZEN; LAMONT, 2002).

Em Algoritmos Genéticos, a solução candidata a solucionar um dado problema pode ser representada por um cromossomo. Deste modo, a representação de um indivíduo (solução) é a primeira etapa na elaboração de um AG, sendo essencial para o desempenho do algoritmo.

Na Seção 2.1.3 são descritos os operadores de reprodução que podem estar presentes em um Algoritmo Genético.

2.1.3 Operadores de Reprodução

2.1.3.1 Mutação

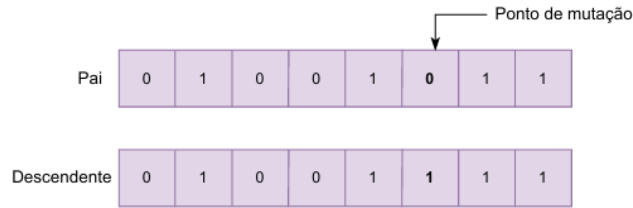
O operador de mutação foi inspirado no mecanismo de mutação gênica presente na teoria da evolução natural, sendo aplicado a um ou mais genes de um cromossomo para modificá-los de maneira aleatória (GABRIEL; DELBEM, 2008).

Em algumas abordagens, a mutação pode imitar a reprodução assexuada, em que um indivíduo sozinho é capaz de gerar um novo descendente. Em outros casos, a mutação é aplicada sobre um novo indivíduo gerado a partir do cruzamento, ou sobre toda a população atual, se a taxa de mutação permitir. Logo, a taxa de mutação é a probabilidade de ocorrer, ou não, a mutação em um gene. Os valores utilizados para tal taxa normalmente são pequenos, uma vez que valores muito altos podem resultar em indivíduos piores do que o original (GABRIEL; DELBEM, 2008).

Entre os operadores de mutação mais populares estão a mutação padrão, mutação uniforme e mutação Gaussiana. No caso de codificação binária (zeros e uns), o operador de mutação padrão consiste simplesmente em inverter o valor de um gene em um cromossomo (GOLDBERG, 1989). Assim, se o alelo de um gene selecionado for 0, após a aplicação da mutação ele se tornará 1.

A Figura 2 mostra como ocorre o processo mencionado.

Figura 2 – Representação do operador de mutação padrão.



No caso de codificação em ponto flutuante os operadores mais populares são a mutação uniforme e mutação Gaussiana.

O primeiro seleciona aleatoriamente um componente $k \in \{1, 2, \dots, n\}$ do cromossomo $x = [x_1, \dots, x_k, \dots, x_n]$ e gera um indivíduo $x' = [x_1, \dots, x'_k, \dots, x_n]$, sendo x'_k um número aleatório (com distribuição de probabilidade uniforme) amostrado no intervalo $[L_I, L_S]$, sendo L_I o limite inferior e L_S o limite superior para o valor do alelo x_k (GABRIEL; DELBEM, 2008).

No caso da mutação Gaussiana, todos os componentes de um cromossomo x são modificados pela seguinte expressão:

$$x' = x + N(0, \sigma)$$

Na qual $N(0, \sigma)$ é um vetor de variáveis aleatórias Gaussianas independentes, com média zero e desvio padrão σ .

2.1.3.2 Recombinação

O operador de recombinação, também chamado de *crossover* ou cruzamento, é inspirado na reprodução sexuada, em que dois indivíduos se combinam para gerar um ou mais descendentes. Este operador promove a troca de características entre os pais durante a reprodução, que são então transmitidas aos descendentes. A ideia do *crossover* é produzir indivíduos com as características potencialmente melhores dos pais. No contexto dos Algoritmos Genéticos, a recombinação é o principal operador de reprodução (GOLDBERG, 1989).

Existem diversos tipos de operadores de cruzamento, sendo os mais comuns o de 1-ponto, n-pontos e a recombinação uniforme. No cruzamento 1-ponto é selecionado aleatoriamente um ponto de corte no cromossomo, dividindo-o em duas partes: uma à direita e outra à esquerda do corte. Cada descendente é formado pela junção da partição à esquerda de um pai com a partição à direita do outro pai.

Na recombinação de n-pontos o cromossomo é dividido em n partes que serão recombinadas. Desta forma, no caso de um cruzamento em 2-pontos o cromossomo será dividido em três partes: uma à esquerda, uma central e outra à direita. A Figura 3b ilustra um exemplo com 2 pontos de corte.

Na recombinação uniforme é criada uma lista de variáveis aleatórias de distribuição uniforme em $[0,1]$. Caso o valor da variável aleatória, para cada posição, seja menor do que um determinado valor P , o gene terá como origem o Pai 1, caso contrário virá do Pai 2. O segundo filho é gerado como o mapeamento oposto do primeiro.

Figura 3 – Recombinação 1-ponto e 2-pontos.

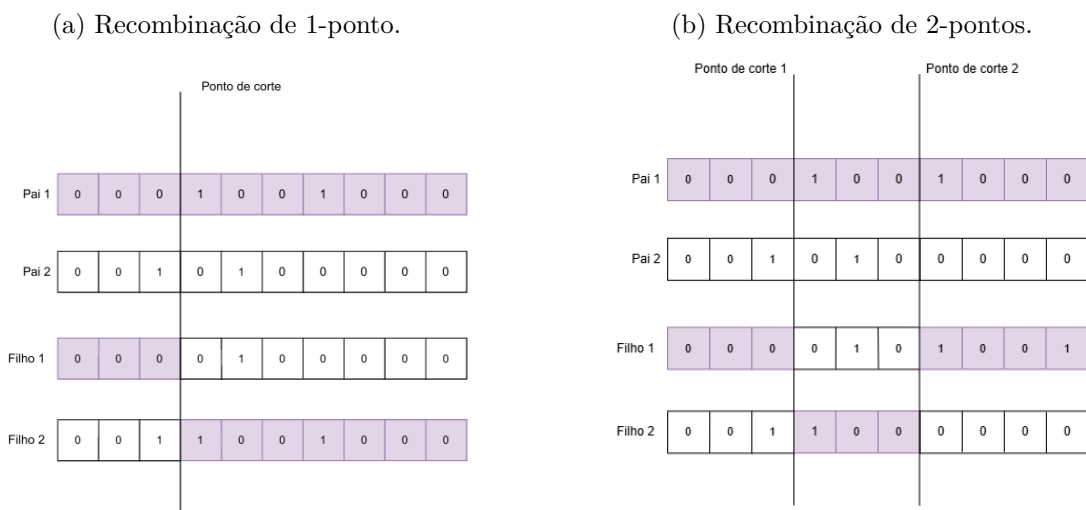
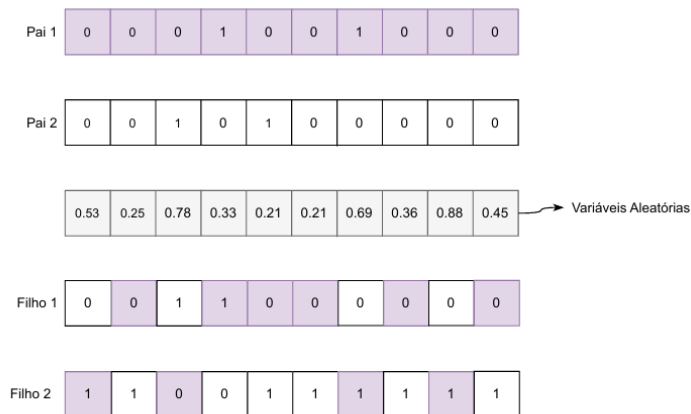


Figura 4 – Recombinação uniforme com $P = 0.5$.



A seguir, serão explicados os principais métodos de seleção dos Algoritmos Genéticos.

2.1.4 Métodos de Seleção

A seleção natural busca selecionar os mais aptos em meio a uma população de indivíduos, que sofrem pressões do ambiente, para perpetuar suas características durante as próximas gerações. Desta forma, o processo de seleção favorece os indivíduos mais bem adaptados; no caso dos algoritmos genéticos, as soluções com melhor *fitness*.

Existem diversos métodos de seleção descritos na literatura, entre os quais os métodos da roleta, seleção elitista, *ranking* e por torneio são amplamente utilizados no desenvolvimento de AGs.

No Método da Roleta (Figura 5), as soluções candidatas de uma população são representadas em uma roleta fictícia, onde cada fatia é o equivalente ao *fitness* relativo de cada indivíduo. Desta forma, as soluções com melhor valor de aptidão têm maior chance de serem escolhidas para gerar descendentes. O indivíduo escolhido é aquele em que o ponteiro da roleta aponta após ela ser girada. Esse processo é repetido n vezes, até que o número de indivíduos necessário seja selecionado. Uma implementação comum deste método é calcular uma lista de valores $[a_1, a_2, \dots, a_\mu]$, tais que $a_i = \sum_1^i P(i)$, em que $P(i)$ é a probabilidade proporcional ao *fitness* de um indivíduo i ser escolhido para realizar a recombinação (GABRIEL; DELBEM, 2008).

No método por torneio, é sorteado um subconjunto da população com k indivíduos, em que os melhores desse grupo são escolhidos para decidir quem irá reproduzir. Por exemplo, em um torneio de 3 ($k = 3$), como mostrado na Figura 6, três indivíduos obtidos aleatoriamente da população competem entre si e o ganhador (melhor *fitness*) torna-se um dos pais (GABRIEL; DELBEM, 2008), sendo o processo repetido para selecionar o outro pai. Na Figura 6, o cromossomo 1 vence ao apresentar o melhor *fitness*, supondo que o objetivo fosse maximizar o valor de aptidão. Nesta estratégia de seleção, quanto maior o k , maior é a pressão seletiva que favorece os mais aptos e quanto maior a pressão, maior o número de indivíduos aptos escolhidos (MILLER; GOLDBERG, 1995).

A seleção baseada em *ranking* utiliza a posição dos indivíduos ordenados de acordo com o *fitness* para determinar a probabilidade de seleção, podendo ser utilizados mapeamentos lineares ou não lineares. Uma maneira de implementar essa estratégia é passar os n melhores indivíduos para a próxima geração (ZUBEN, 2000).

Por fim, a seleção elitista é uma estratégia que busca preservar os melhores indivíduos, mantendo-os na população. Esse método, também chamado de elitismo, simplesmente perpetua sempre uma taxa dos melhores indivíduos da geração atual na geração seguinte (FOGEL, 1994; MICHALEWICZ, 1996).

Figura 5 – Método da Roleta.

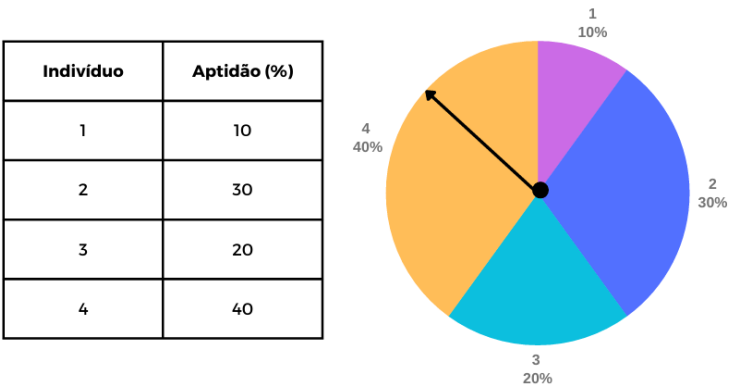
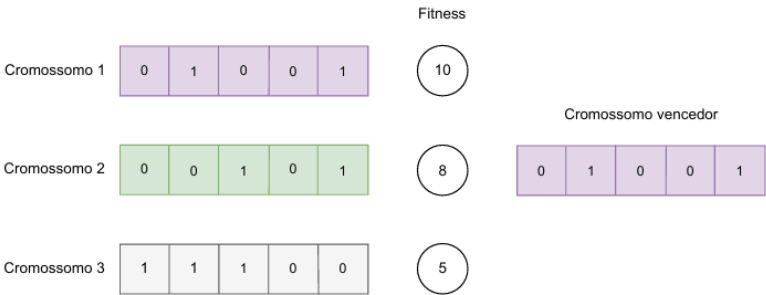


Figura 6 – Método de Torneio.



2.2 Teoria dos Jogos e o Problema do Quebra-Cabeça de 8 peças

A Teoria dos Jogos é um ramo da matemática aplicada, mas que atualmente serve como base de referência para estudos de disciplinas como ciências políticas, linguística, economia, ciências da computação, entre outras (LEYTON-BROWN; SHOHAM, 2008). Desde 1970, esta teoria também é utilizada na biologia para estudar o comportamento animal e a evolução genética por seleção natural, sendo de grande relevância para compreender e prever o desfecho da evolução de certas espécies (SMITH, 1982).

Segundo Leonard (1995), a teoria foi criada há mais de 70 anos por John Von Neumann e Oskar Morgenstern quando publicaram o livro “*The Theory of Games and Economic Behaviour*” (NEUMANN; MORGENSTERN, 1944), sendo aprofundada ao longo dos anos. De modo geral, a Teoria dos jogos pode ser definida como a teoria dos modelos matemáticos que estuda a escolha de decisões ótimas sob condições de conflito (SARTINI et al., 2004). Nela é estudada a interação entre agentes inteligentes em um ambiente competitivo, de modo que os jogadores tentam fazer as decisões corretas, dentro do limite das regras do jogo, para ganhar (MAGALHAES; Sá, 2015).

O problema do quebra-cabeça de 8 peças, ou 8-puzzle, pode ser relacionado à teoria dos jogos ao ser modelado como um jogo de estratégia para um jogador, onde o objetivo é encontrar a sequência ótima de movimentos para resolver o desafio. Neste contexto, o jogador atua como um agente que toma decisões estratégicas, similar ao que ocorre em jogos de soma zero na teoria dos jogos, onde os ganhos de um jogador correspondem às perdas do outro, ou seja, um jogador só pode ganhar se o outro perder (SANTOS et al., 2018). No 8-puzzle, embora seja um jogo de um único jogador, o adversário pode ser visto como o estado desordenado das peças e cada decisão tomada pode impactar o estado do quebra-cabeça de maneira positiva ou negativa, afetando o resultado final da resolução.

O jogo do quebra-cabeça de 8 peças foi criado por Sam Loyd na década de 1870 e consiste em um jogo de tabuleiro, ou matriz 3x3, de blocos deslizáveis, que possui 8 peças numeradas de 1 a 8 e uma peça em branco. O quebra-cabeça começa com uma disposição embaralhada das peças, e o objetivo do jogo é movê-las para alcançar o estado final, onde todos os números estão dispostos em ordem crescente da esquerda para a direita e de cima para baixo (QIAN, 1995). De modo geral, o objetivo do 8-puzzle é reorganizar uma configuração inicial das 8 peças quadradas de um tabuleiro 3x3 em uma configuração final previamente definida, deslizando sucessivamente as peças para o quadrado vazio (peça em branco) (SHABAN; ALKALLAK; SULAIMAN, 2010).

Solucionar o problema envolve determinar os movimentos necessários para atingir o estado objetivo a partir de um estado inicial. Em algumas versões do jogo, um dos grandes desafios é chegar à configuração final com o menor número de movimentos possível (IGWE; PILLAY; RAE, 2013).

Um exemplo do quebra-cabeça é ilustrado na Figura 7.

Figura 7 – Quebra-cabeça deslizante de 8 peças.

Estado Inicial			Estado Objetivo		
1	2	3		1	2
8		4	3	4	5
7	6	5	6	7	8

2.3 Trabalhos Relacionados

Esta seção tem como objetivo apresentar trabalhos da literatura, no qual seus autores buscaram aplicar algoritmos genéticos para solucionar problemas de diferentes tipos de jogos.

Suh e Lee (1989) propuseram um algoritmo genético, diferente dos outros métodos até então propostos, para solucionar o problema dos quebra-cabeças deslizantes. O algoritmo em questão utiliza um esquema de representação orientado a operadores, onde cada movimento do jogo é representado como $((x, y), o)$, em que (x, y) representa a localização do estado inicial e " o " representa os movimentos. A abordagem feita pelos autores utiliza de operadores genéticos de mutação e *crossover*. Os parâmetros de ajuste para o algoritmo foram o tamanho inicial, tamanho mínimo, tamanho máximo, geração máxima, população, taxa de cruzamento e taxa de mutação. O algoritmo proposto conseguiu solucionar o problema com resultados satisfatórios para os quebra-cabeças 3x3 e 4x4. Entretanto, percebeu-se que o AG teve problemas para resolver quebra-cabeças de blocos deslizantes que exige uma sequência de movimentos muito grande.

Shaban, Alkallak e Sulaiman (2010) propuseram uma heurística para solucionar o quebra-cabeça de 8 peças, em que os indivíduos são criados aleatoriamente, a fim de gerar uma população inicial. O tamanho da população é equivalente ao número de peças fora do lugar. Para os operadores genéticos, foram utilizados a seleção por torneio, o método dos cromossomos ordenados para o *crossover*, e o método de mudança de ordem para a mutação. O *fitness* é calculado para cada cromossomo e comparado com a função objetivo. Segundo os autores, a utilização de algoritmos genéticos para a solução do 8-puzzle é viável, uma vez que ele consegue encontrar a solução em um grande espaço de busca.

Junior e Guimaraes (2011) fizeram uma análise da solução do 8-puzzle utilizando algoritmo genético e *backtracking*. Para o AG foram utilizados os operadores genéticos de mutação a 60%, recombinação a 30%, e o método do elitismo para a seleção. Os algoritmos genéticos mostraram ser mais eficientes que o *backtracking*, encontrando soluções com menos movimentos e em menos tempo.

Em Schreiber (FONTOURA; PACCE; SCHREIBER, 2012) a heurística proposta utiliza de um algoritmo genético que, dada uma configuração do tabuleiro com indicação do posicionamento de todos os veículos, no jogo *Rush Hour*, gera a sequência de movimentos necessários para retirar o veículo do jogador através da saída indicada. Cada indivíduo é representado por uma matriz 6x6, com números inteiros, onde cada célula da matriz representa uma posição no tabuleiro. A população inicial não é gerada aleatoriamente, mas sim por todos os possíveis tabuleiros gerados com apenas um movimento. A partir destes, gera-se em seguida todos os tabuleiros possíveis com um segundo movimento, formando

uma população inicial composta por configurações alcançáveis em até dois movimentos a partir do estado inicial, o que permite ao algoritmo já conter soluções próximas da ideal desde o início.

Os autores concluíram que o procedimento heurístico proposto foi capaz de resolver todos os tabuleiros, independentemente do nível de complexidade. A heurística proposta mostrou-se adequada para aplicações reais, pois é capaz de produzir boas soluções em tempo computacional reduzido.

Bezulj e Jovicic (BEZULJ; JOVICIC, 2018) destacaram a importância de métricas de compatibilidade e algoritmos genéticos para a resolução automática de quebra-cabeças de imagem. O algoritmo utiliza seleção, reprodução e mutação para encontrar soluções, com foco na função de *fitness* e no operador de recombinação. Existem três fases para o *crossover* e o artigo em questão propõe modificar o operador e alterar a métrica de compatibilidade, o que resultou em um aumento de 8% na precisão média.

Yang (YANG et al., 2023) examinou como o AG pode ser aplicado ao problema do 8-puzzle, ao modificar determinados parâmetros. Os resultados mostram que, ao aumentar a taxa de cruzamento e a taxa de mutação, o problema é resolvido mais rapidamente, mas um aumento do tamanho da população pode levar a tempos de solução mais longos.

Bukhori, Felix e Saddam (BUKHORI; FELIX; ALI, 2024) fizeram um compilado de análises a respeito da utilização de algoritmos genéticos para solucionar problemas de quebra-cabeças. Concluiu-se que jogos como os de peças deslizantes, sudoku, jogo da velha e quebra-cabeças de imagem apresentam padrões similares de solução e podem ser executados por AGs com alto grau de acurácia. A grande relação entre esses jogos é o fato de utilizarem de operadores de mutação, recombinação e seleção e seus cromossomos serem representados por matrizes ou vetores.

Diante dos trabalhos apresentados, percebe-se que os algoritmos genéticos podem ser empregados na solução de diversos problemas de jogos de tabuleiro e quebra-cabeças. Em vista disto, a análise dos trabalhos relacionados revelou importantes contribuições para o entendimento do tema. Estudos como os de (SHABAN; ALKALLAK; SULAIMAN, 2010) e (YANG et al., 2023) foram utilizados como base para o desenvolvimento deste trabalho, uma vez que forneceram referências teóricas essenciais sobre a aplicação de AGs em jogos como o 8-puzzle.

2.4 Considerações Finais

Na área da computação, a resolução de problemas em jogos de tabuleiro pode ser efetivamente abordada por meio da aplicação de Algoritmos Genéticos. Neste capítulo, foram discutidos conceitos fundamentais sobre algoritmos genéticos, com foco na

sua aplicação em jogos como o 8-puzzle. Além disto, foram analisados artigos científicos que exploram o uso de AGs para resolver e avaliar o desempenho em jogos de tabuleiro e quebra-cabeças. Estes estudos destacam a importância da escolha adequada dos parâmetros e dos ajustes nos operadores genéticos, mostrando como essas decisões podem impactar significativamente o desempenho do algoritmo, influenciando os resultados obtidos.

3 Implementação do Algoritmo Genético para o problema 8-puzzle

Neste capítulo, será apresentada a implementação do Algoritmo Genético proposto neste Trabalho de Conclusão de Curso. Desta forma, serão expostas: a representação do indivíduo, a inicialização da população, a função de aptidão e as operações de seleção, recombinação e mutação, específicas para o problema.

O problema do 8-puzzle consiste em mover peças numeradas em uma matriz 3×3 até atingir uma configuração alvo (estado objetivo), partindo de uma configuração inicial. O objetivo é minimizar o número de gerações e os movimentos necessários para atingir a solução do problema.

Neste trabalho, o Algoritmo Genético (AG) foi estruturado em três classes principais: *Pad*, *Chromosome* e *Solver*. A classe *Pad* modela o estado do tabuleiro, enquanto a classe *Chromosome* representa uma sequência de movimentos que compõem uma solução potencial para o quebra-cabeça. Por fim, a classe *Solver* encapsula os procedimentos utilizados pelo Algoritmo Genético. A seguir, são apresentadas as principais classes e funções do AG desenvolvido neste trabalho.

3.1 Desenvolvimento do Algoritmo Genético

O AG foi desenvolvido em linguagem Java no sistema operacional Windows 10 de um notebook Dell Inspiron 5548 com 16 GB de memória RAM, tendo como referência o algoritmo desenvolvido por (YANG et al., 2023). O código do AG implementado neste Trabalho de Conclusão de Curso está disponível em: <https://github.com/laurarmuniz/8_puzzle>.

3.2 Classe Pad: Representação do Tabuleiro

A classe *Pad* contém as principais funções que representam o estado do tabuleiro do quebra-cabeça de 8 peças. Inicialmente, o tabuleiro é retratado como uma matriz 3×3 , inicializada com valores de 0 a 8, sendo zero o espaço vazio. Após a inicialização da matriz, é chamada a função *embaralhar*, responsável por embaralhar as peças do quebra-cabeça, que antes estavam posicionadas em ordem crescente.

Entre as funções de movimento que compoem a classe *Pad* estão: *moverParaEsquerda*, *moverParaDireita*, *moverParaCima* e *moverParaBaixo*. Estas funções permitem

o deslocamento de uma peça em direção ao espaço vazio, respeitando a restrição de que apenas peças adjacentes ao zero podem ser movidas. As funções de movimento utilizam um método auxiliar chamado *encontraIndex*, responsável por encontrar um determinado valor passado como parâmetro da função. No caso das funções de movimento, o método *encontraIndex* é utilizado para encontrar a posição do espaço vazio no tabuleiro.

Após a verificação da localização do espaço vazio, é verificado se o número 0 pode se mover para uma das quatro direções válidas (cima, baixo, esquerda, direita); por exemplo, na função *moverParaEsquerda* o código irá verificar se o zero está na primeira coluna. Se o espaço vazio estiver nesta posição e não for capaz de mover para a esquerda, então o código retornará falso e tentará movê-lo para outra direção.

O mesmo é feito para as outras três funções de movimento. A única diferença é que as funções *moverParaCima* e *moverParaBaixo* verificam a posição do espaço vazio em relação às linhas e a função *moverParaDireita*, assim como a *moverParaEsquerda*, verifica a posição do zero em relação à coluna. Outra função importante para o movimento das peças é a função *trocar*, que faz a troca de dois valores no tabuleiro.

A função *custoManhattan*, na classe *Pad*, mede o quão distante o estado atual do tabuleiro está do estado final. Esta função, é baseada no cálculo da distância de Manhattan (métrica usada para determinar a distância entre dois pontos em um caminho semelhante a uma grade) (DEZA; DEZA, 2009). Na função implementada, é estabelecido um dicionário referência que contém as posições desejadas de cada bloco no tabuleiro. A função avalia o quão longe o estado atual do quebra-cabeça está da solução final. Para cada peça, a distância de Manhattan é a soma dos deslocamentos horizontal e vertical necessários para levar a peça até sua posição correta.

O cálculo da distância de Manhattan é expressado da seguinte forma:

$$d_{\text{Manhattan}} = |x' - x''| + |y' - y''| \quad (3.1)$$

$d_{\text{Manhattan}}$: Distância total entre a posição atual de um bloco e sua posição final no tabuleiro, somando deslocamentos horizontal e vertical.

x' : Coordenada x (linha) atual da peça no tabuleiro, obtida pela função que localiza a peça no estado atual.

x'' : Coordenada x (linha) desejada da peça no tabuleiro, definida no dicionário de referência.

y' : Coordenada y (coluna) atual da peça no tabuleiro.

y'' : Coordenada y (coluna) desejada da peça no tabuleiro.

Desta forma, é atribuído um valor menor para estados mais próximos do estado final desejado e caso o tabuleiro esteja perto da solução, o custo será próximo de 0.

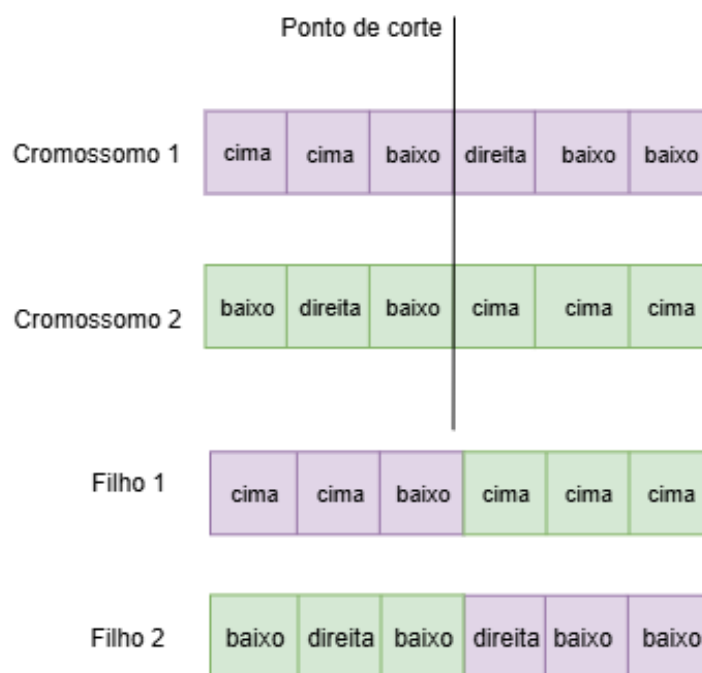
3.3 Classe Chromosome: Representação do Indivíduo

A classe *Chromosome* é responsável por representar uma sequência de movimentos que compõem uma solução potencial para o quebra-cabeça. Nesta classe foram implementados métodos auxiliares de *crossover* e mutação, assim como um método responsável pela atualização do *fitness* de cada indivíduo.

A função de *crossover* realiza a reprodução de dois cromossomos, ordenando-os previamente para priorizar o cromossomo de melhor *fitness*. Um ponto de corte é definido, dividindo os genes de ambos os pais em duas partes. Em seguida, dois novos cromossomos são gerados: o primeiro filho combina a primeira metade do melhor pai com a segunda metade do segundo pai, enquanto o segundo filho combina a primeira metade do segundo pai com a segunda metade do melhor pai. Por fim, os filhos são retornados como novos objetos da classe *Chromosome*, sendo eles parte da próxima geração do AG.

Um exemplo de funcionamento da função de recombinação é ilustrado da Figura 8. O ponto de corte é definido como o tamanho do menor cromossomo dividido por dois. Como, neste exemplo, ambos os cromossomos são do mesmo tamanho, o ponto de corte é definido com o valor igual a $6/2$, ou seja, 3.

Figura 8 – Representação do funcionamento da função de *crossover*.



A função de mutação modifica o cromossomo atual, alterando ou adicionando movimentos para promover diversidade na população. Inicialmente, é calculada a chance de adicionar um novo movimento ou modificar um existente, priorizando a adição caso o cromossomo esteja vazio. Se a decisão for adicionar um movimento, um novo movimento é escolhido aleatoriamente dentre as opções disponíveis, com verificação para evitar movimentos que desfaçam o último passo do cromossomo. Caso contrário, ao modificar um movimento existente, um índice aleatório do cromossomo é substituído por um novo movimento, garantindo que ele seja diferente do original. Por fim, a função atualiza o *fitness* do cromossomo, refletindo as mudanças realizadas. Um exemplo de funcionamento da função de mutação é ilustrado na Figura 9 e na Figura 10.

Figura 9 – Função de mutação - modificação de movimento no cromossomo atual.

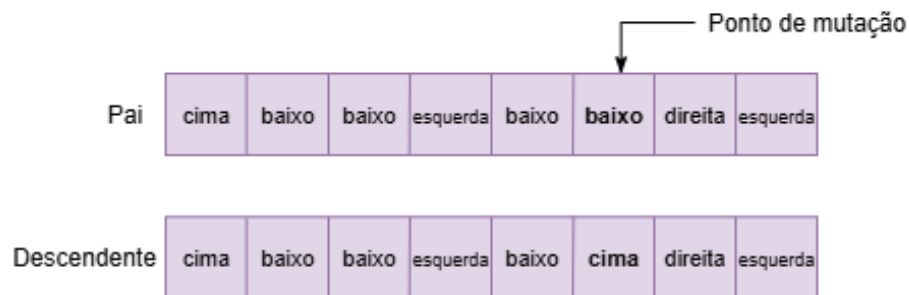
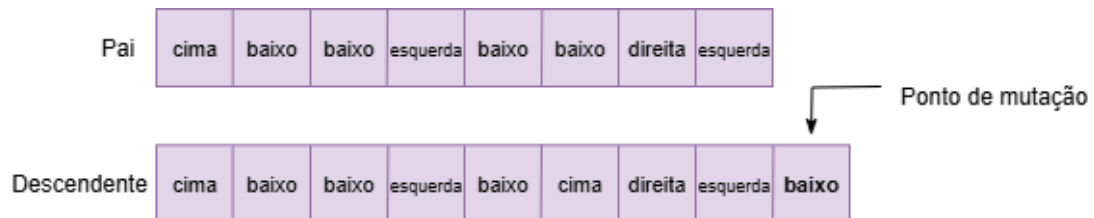


Figura 10 – Função de mutação - adição de movimento no cromossomo atual.



A função *updateFitness* é responsável por recalcular o valor de *fitness* do cromossomo, medindo sua adequação à solução do problema. Primeiro, ela cria uma cópia do tabuleiro inicial e aplica cada movimento do cromossomo nessa cópia, seguindo a ordem dos movimentos. O custo da solução é calculado como a distância do estado atual em relação ao objetivo (estado resolvido do quebra-cabeça) utilizando a função *custoManhattan*, e uma penalidade é adicionada proporcional ao tamanho do cromossomo para desencorajar soluções com movimentos excessivos. Tal penalidade é representada no código pela variável *errorGeneLen*, sendo ela o tamanho do cromossomo multiplicado por 0,0001.

Em seguida, o estado atual do tabuleiro é comparado ao estado objetivo, representado por um tabuleiro resolvido. Caso o tabuleiro esteja resolvido corretamente, o *fitness* é ajustado para o menor valor possível (0), indicando que essa solução é preferível. Caso contrário, o *fitness* é calculado como a soma da penalidade pelo comprimento do cromossomo, juntamente com o custo da distância do tabuleiro atual em relação ao objetivo, que

nada mais é do que a distância de Manhattan calculada sobre uma cópia do tabuleiro. O valor do custo é explicitado nos cálculos a seguir:

$$\text{errorGeneLen} = \text{cromossomo.size}() * 0,0001 \quad (3.2)$$

$$\text{distanciaCusto} = \text{temp.custoManhattan}() \quad (3.3)$$

$$\text{fitness} = \text{distanciaCusto} + \text{errorGeneLen} \quad (3.4)$$

3.4 Classe Solver: Solução do quebra-cabeça

A classe *Solver* é responsável por encapsular a lógica do Algoritmo Genético. Nesta classe são implementadas funções para inicialização da população, cálculo de aptidão, elitismo, seleção, recombinação e mutação. Tais funções serão melhor explicadas em subseções a seguir.

3.4.1 Inicialização da População

O método de inicialização da população (*initPopulacao*) é responsável por criar a população inicial de cromossomos. Cada cromossomo representa uma sequência de movimentos que podem ser realizados no tabuleiro do 8-puzzle. A inicialização ocorre de forma aleatória, garantindo diversidade na população e possibilitando explorar, de forma eficiente, o espaço de soluções.

Para cada cromossomo, é gerado um número aleatório de movimentos, com um comprimento entre 15 e 40 genes. Cada gene é escolhido aleatoriamente a partir do conjunto de movimentos válidos (cima, baixo, esquerda e direita). Estes movimentos são aplicados ao espaço vazio do tabuleiro, também identificado pelo número zero na implementação do AG.

3.4.2 Cálculo de aptidão (*Fitness*)

A função *calculaFitness* avalia a qualidade de cada cromossomo na população, determinando quão próximo ele está de resolver o problema do 8-puzzle. Esta avaliação é realizada por meio do método *updateFitness* de cada cromossomo, que calcula o valor do *fitness* com base em duas métricas principais: a distância até o estado final esperado e o comprimento do cromossomo, conforme explicado na Seção 3.3.

Se um cromossomo resolve completamente o tabuleiro (atingindo o estado objetivo esperado), ele é identificado como o melhor possível, com um valor de *fitness* menor que o destaca dos demais.

Após calcular a aptidão de cada cromossomo, a função verifica qual deles tem o menor valor de *fitness* e o armazena como o melhor da geração. Além disso, a função calcula o *fitness* médio de toda a população, o que possibilita uma melhor análise do desempenho atual do algoritmo.

3.4.3 Seleção e elitismo

A seleção de indivíduos para a próxima geração foi implementada com base no método da roleta. Desta forma, o objetivo desta função é selecionar dois cromossomos para participar das próximas etapas genéticas (recombinação e mutação). O processo é repetido duas vezes para escolher dois indivíduos distintos:

1. É atribuído um peso a cada cromossomo, inversamente proporcional ao seu *fitness* uma vez que se deseja minimizá-lo, para que os mais aptos tenham maior probabilidade de serem escolhidos. A lógica é que quanto menor for o *fitness*, maior a chance do cromossomo ser escolhido.
2. Uma lista de pesos acumulados é construída para facilitar o sorteio.
3. Um número aleatório é gerado e comparado com os valores acumulados. Se o número aleatório for menor que o valor acumulado, escolhemos este cromossomo.

Deste modo, a função retorna os dois cromossomos escolhidos, os pais, que atuarão em seguida na operação de reprodução.

Quanto ao método que implementa o elitismo, nele é selecionado um percentual da população (taxa de elitismo) com base no valor de *fitness* de cada indivíduo, priorizando os cromossomos com melhores desempenhos. Estes cromossomos são mantidos na nova geração sem alterações. Em seguida, a função preenche o restante da população com os indivíduos gerados com os operadores de reprodução, e por fim, a população é atualizada para incluir tanto os indivíduos elitistas quanto os novos selecionados.

A seguir, serão explicados os operadores de reprodução do AG para o problema do quebra-cabeça com 8 peças.

3.4.4 Recombinação

A função de recombinação (*crossover*), presente na classe *Solver*, é responsável por combinar os genes de pares de cromossomos, gerando novos descendentes. Inicialmente,

verifica-se se a população atual contém pelo menos dois cromossomos; caso contrário, o cruzamento não é realizado, e a função é encerrada. O número de cruzamentos a ser executado é determinado com base na taxa de *crossover* (*txCrossover*), que define a probabilidade de ocorrer o cruzamento entre aqueles pais. Para cada cruzamento, dois cromossomos são selecionados como pais utilizando a técnica de seleção por roleta, implementada pela função *selecaoPorRoleta*. Em seguida, os pais são combinados pela função *Chromosome.crossOver*, que gera dois descendentes, se assim for o caso (ou seja, se for aplicado o *crossover*). Após a atualização do *fitness* destes novos cromossomos, eles são adicionados diretamente à população, promovendo a diversidade genética e possibilitando a evolução do conjunto de soluções.

3.4.5 Mutação

O método de mutação presente na classe *Solver* é responsável por decidir quando cada cromossomo sofrerá alterações. Após a formação da nova população — resultante do processo de seleção e reprodução — o método percorre todos os cromossomos dessa população e, para cada um deles, gera um valor aleatório. Se esse valor for menor que a taxa de mutação definida, o método de mutação da classe *Chromosome* é chamado, aplicando a alteração no cromossomo. Assim, a mutação não ocorre automaticamente no momento da criação de um filho, mas sim posteriormente, sobre qualquer indivíduo da população atual, podendo atingir tanto cromossomos recém-gerados quanto outros já presentes.

3.4.6 Solucionador

A função *solve* é o ponto central da classe *Solver*, responsável por conduzir o Algoritmo Genético até encontrar uma solução satisfatória ou atingir o limite de iterações definido. Ela organiza as etapas principais do algoritmo e garante que a busca por uma solução seja feita de forma eficiente.

O processo tem início com a criação da população inicial, onde os cromossomos são gerados de forma aleatória pela função *initPopulacao*. Em seguida, o algoritmo entra em um ciclo de iterações, onde as soluções são exploradas e aprimoradas por meio da reprodução. O processo de iteração ocorre até que o número máximo de iterações seja atingido ou até que o melhor cromossomo encontre uma solução que satisfaça o critério de erro ótimo (seja igual a zero).

Durante cada iteração, são realizadas etapas importantes, como: o cruzamento de indivíduos para gerar novas combinações genéticas, o cálculo do *fitness* dos cromossomos para avaliar a qualidade das soluções, a aplicação do elitismo para garantir a preservação de uma taxa dos melhores indivíduos na população, e, por fim, a mutação, que promove

a diversidade, evitando estagnação no processo evolutivo. Além disto, a função exibe os resultados de cada iteração por meio da função *display*, permitindo acompanhar a evolução do algoritmo. Ao final, o melhor cromossomo encontrado durante o processo é retornado como solução.

3.5 Considerações Finais

Neste capítulo, foi apresentada a implementação do Algoritmo Genético para resolver o problema do 8-puzzle. O algoritmo foi dividido em três classes principais: a classe *Pad*, que representa o tabuleiro; a classe *Chromosome*, que representa as soluções possíveis; e a classe *Solver*, que controla o processo evolutivo.

Foram implementadas as principais etapas do algoritmo, como a inicialização da população, cálculo do *fitness*, seleção por roleta, elitismo, recombinação (*crossover*) e mutação. Essas etapas trabalham juntas para buscar soluções cada vez melhores ao longo das gerações.

A implementação foi feita de forma clara e organizada, facilitando a compreensão e o ajuste dos parâmetros do algoritmo. Essa base permitiu realizar os experimentos que serão apresentados no próximo capítulo, onde serão analisados os resultados obtidos com diferentes configurações.

4 Resultados

Neste capítulo são apresentados os experimentos realizados com o AG aplicado ao problema do quebra-cabeça de 8 peças, bem como os resultados obtidos. O AG foi executado 20 vezes para cada configuração de parâmetros, sendo os experimentos divididos em três etapas: experimentos com a taxa de mutação, experimentos com taxa de elitismo e experimentos com taxa de *crossover*, respectivamente, a fim de obter a melhor configuração para este problema. A eficiência de um operador e a configuração de parâmetros foram avaliados pela média das gerações obtidas (número de iterações levadas para encontrar a solução do problema), assim como pela média do tamanho de cada cromossomo (número de movimentos realizados). Além disto, também foi avaliada a quantidade de vezes em que o algoritmo encontrou a solução do problema para cada configuração.

4.1 Experimentos com a taxa de mutação

Nas Tabelas 1, 2, 3, 4, e 5, são mostrados os resultados dos testes realizados para a análise do comportamento do Algoritmo Genético desenvolvido para solucionar o problema do quebra-cabeça de 8 peças. Nesta etapa de testes, foram utilizados como parâmetros: população (200); número máximo de gerações (2000); elitismo (1%); taxa de *crossover* (100%), variando a taxa de mutação.

Tabela 1 – Taxa de mutação = 1%.

Execução	Geração	n° Movimentos
1	1	19
2	2001	31
3	1	31
4	81	34
5	1	40
6	7	32
7	2001	19
8	1	35
9	5	30
10	1	40
11	1	28
12	18	40
13	1	29
14	12	39
15	4	33
16	1	20
17	1	21
18	2001	18
19	1	22
20	1	33
Média	8	30
Desvio Padrão	19	7

Para a taxa de mutação igual a 1% (Tabela 1), o AG não conseguiu encontrar a solução para o problema em três execuções (onde o número de gerações alcança 2001). O número de gerações médias em que a solução foi encontrada foi igual a 8, apresentando um desvio padrão populacional igual a 19. A coluna referente ao número de movimentos representa a quantidade de genes (movimentos possíveis das peças do quebra-cabeça) presentes no cromossomo escolhido como solução final (aquele com melhor *fitness*).

Tabela 2 – Taxa de mutação = 3%.

Execução	Geração	n° Movimentos
1	2001	37
2	4	20
3	1	26
4	2	21
5	1	38
6	1	15
7	5	26
8	17	17
9	5	21
10	2001	19
11	1	22
12	1	28
13	2001	15
14	1	19
15	4	42
16	6	33
17	3	25
18	39	38
19	16	36
20	2001	17
Média	7	26
Desvio Padrão	10	8

Para a taxa de mutação igual a 3% (Tabela 2), o AG não conseguiu encontrar a solução para o problema em quatro execuções, sendo um pouco pior que com a taxa de 1%. O número de gerações médias em que a solução foi encontrada foi igual a 7, apresentando um desvio padrão populacional igual a 10. Em relação ao número de movimentos do indivíduo, o AG apresentou uma média igual a 26 movimentos.

Tabela 3 – Taxa de mutação = 5%.

Execução	Geração	n° Movimentos
1	3	32
2	1	16
3	1	36
4	6	40
5	4	34
6	7	35
7	1	40
8	2001	18
9	2001	38
10	1	37
11	2001	32
12	2001	36
13	5	35
14	2001	15
15	1	20
16	5	23
17	2001	33
18	1	39
19	2001	20
20	23	40
Média	5	31
Desvio Padrão	6	8

Para a taxa de mutação igual a 5% (Tabela 3), houveram sete execuções em que o AG não conseguiu encontrar a solução para o problema, sendo bem pior que as anteriores. O número de gerações médias em que a solução foi encontrada foi igual a 5, apresentando um desvio padrão populacional igual a 6 e a média do número de movimentos dos indivíduos foi igual a 31.

Tabela 4 – Taxa de mutação = 15%.

Execução	Geração	n° Movimentos
1	2001	20
2	4	30
3	7	37
4	2001	15
5	3	31
6	1	24
7	4	29
8	1	18
9	1	37
10	1	16
11	2001	16
12	1	20
13	4	40
14	1	29
15	5	23
16	1	19
17	3	22
18	1	31
19	3	22
20	27	27
Média	4	25
Desvio Padrão	6	7

Para a taxa de mutação igual a 15% (Tabela 4), houveram três execuções em que o AG não conseguiu encontrar a solução para o problema, com um resultado semelhante à taxa de 1%. O número de gerações médias em que a solução foi encontrada foi igual a 4, apresentando um desvio padrão populacional igual a 6. A média do número de movimentos dos indivíduos foi igual 25. Tanto o resultado do número de movimentos quanto dos valores de *fitness* ficaram melhores neste caso do que com 1%.

Tabela 5 – Taxa de mutação = 30%.

Execução	Geração	n° Movimentos
1	2001	17
2	1	16
3	1	36
4	10	37
5	1	20
6	2001	27
7	1	25
8	1	28
9	11	23
10	2001	23
11	6	65
12	1	36
13	4	30
14	3	32
15	11	35
16	12	79
17	6	32
18	1	29
19	2001	15
20	1	16
Média	4	31
Desvio Padrão	4	15

Para a taxa de mutação igual a 30% (Tabela 5), houveram quatro execuções em que o AG não encontrou a solução para o problema, semelhante ao teste com a taxa de 3%. O número de gerações médias em que a solução foi encontrada foi igual a 4, apresentando um desvio padrão populacional igual a 4. O número médio de movimentos encontrado para os indivíduos foi igual a 31 com desvio padrão de 15.

4.1.1 Análise dos resultados

Os experimentos realizados com diferentes taxas de mutação permitiram observar de forma clara como este parâmetro influencia o desempenho do Algoritmo Genético na resolução do problema do 8-puzzle. Considerando apenas a qualidade das soluções — isto é, desconsiderando o tempo de execução — os resultados indicam que a taxa de mutação igual a 15% apresentou o melhor desempenho geral, alcançando a solução ótima em 17 de 20 execuções.

Esta taxa produziu soluções com menor média do número de movimentos (25 movimentos) e apresentou baixa variação entre as execuções, demonstrando eficiência e

consistência na convergência para soluções consideradas boas e curtas.

As taxas mais altas, como 30%, embora ainda encontrem boas soluções, apresentaram maior instabilidade no número de movimentos das trajetórias e mais dispersão nos resultados, o que compromete a consistência.

Portanto, a taxa de mutação de 15% se destacou como a mais equilibrada entre qualidade da solução final, consistência dos resultados e capacidade de encontrar o estado objetivo com maior frequência. Estes resultados reforçam a importância da escolha adequada deste parâmetro, especialmente em problemas onde a qualidade da solução é relevante para resolver o problema.

Segue tabela comparativa dos resultados dos experimentos realizados para a taxa de mutação (Tabela 6).

Tabela 6 – Resumo comparativo entre as taxas de mutação.

Taxa	% Soluções ótimas	n° Movimentos (média)	Geração (média)
1%	85%	30	8
3%	80%	26	7
5%	70%	31	5
15%	85%	25	4
30%	80%	31	4

4.2 Experimentos com a taxa de elitismo

Nesta seção, foram realizados os experimentos relacionados a taxa de elitismo utilizada na implementação do Algoritmo Genético desenvolvido neste trabalho. Nas Tabelas 7, 8, 9 e 10, são expostos os dados obtidos dos testes realizados para a análise do comportamento do AG para diferentes taxas de elitismo. Nesta etapa, foram utilizados como parâmetros: população (200); número máximo de gerações (2000); taxa de mutação (15%); taxa de *crossover* (100%).

Tabela 7 – Taxa de elitismo = 1%.

Execução	Geração	n° Movimentos
1	2001	31
2	2001	20
3	5	35
4	3	25
5	2001	18
6	1	19
7	4	23
8	3	31
9	2001	41
10	1	33
11	2001	32
12	1	29
13	13	41
14	1	20
15	1	17
16	2001	15
17	4	40
18	7	21
19	2	38
20	1	15
Média	3	27
Desvio Padrão	3	9

A Tabela 7 apresenta os resultados obtidos ao utilizar a taxa de elitismo igual a 1% para a resolução do problema do 8-puzzle. Foram realizadas 20 execuções, e observou-se que, em média, as soluções foram encontradas em apenas 3 gerações, com um tamanho médio de solução de 27 movimentos e desvio padrão de 9. Para esta execução de testes, o algoritmo encontrou a solução ótima para 14 das 20 execuções.

Tabela 8 – Taxa de elitismo = 5%.

Execução	Geração	n° Movimentos
1	2001	39
2	2001	17
3	1	20
4	1	19
5	4	38
6	2001	15
7	2	18
8	2	29
9	2001	20
10	273	26
11	1592	30
12	3	20
13	9	40
14	4	32
15	2001	22
16	9	16
17	4	42
18	1	24
19	2001	29
20	4	19
Média	136	26
Desvio Padrão	410	8

Para a taxa de elitismo igual a 5% (Tabela 8), o AG encontrou a solução ótima do problema em 14 das 20 execuções realizadas. O número médio de gerações necessárias foi igual a 136, com desvio padrão populacional de 410. O número médio de movimentos encontrado para os indivíduos foi de 26 movimentos.

Tabela 9 – Taxa de elitismo = 15%.

Execução	Geração	n° Movimentos
1	2001	20
2	2001	20
3	134	22
4	1	38
5	77	24
6	4	30
7	1	16
8	7	30
9	1	25
10	3	35
11	1	30
12	3	38
13	1	18
14	3	38
15	2001	28
16	2001	19
17	7	37
18	3	30
19	495	15
20	1	38
Média	44	28
Desvio Padrão	118	8

Para a taxa de elitismo igual a 15% (Tabela 9), o AG encontrou a solução ótima do problema em 16 das 20 execuções realizadas, sendo um resultado melhor que as anteriores neste quesito. O número médio de gerações necessárias foi igual a 44, com desvio padrão populacional de 118. O número médio de movimentos encontrado para os indivíduos foi igual a 28.

Tabela 10 – Taxa de elitismo = 20%.

Execução	Geração	n° Movimentos
1	4	31
2	3	35
3	1	25
4	1	24
5	3	27
6	4	21
7	2	38
8	1	37
9	1	24
10	4	31
11	2001	16
12	2	20
13	502	44
14	12	38
15	2	32
16	2001	20
17	1	26
18	1	37
19	4	29
20	577	16
Média	63	29
Desvio Padrão	169	8

Para a taxa de elitismo igual a 20% (Tabela 10), o AG encontrou a solução ótima do problema em 18 das 20 execuções realizadas, melhor resultado obtido neste quesito até o momento. O número médio de gerações necessárias foi igual a 63, com desvio padrão populacional de 169. O número médio de movimentos encontrado para os indivíduos foi igual a 29.

4.2.1 Análise dos resultados

Os experimentos realizados com diferentes taxas de elitismo demonstraram a influência deste parâmetro no desempenho do Algoritmo Genético para o problema do 8-puzzle. Considerando a qualidade das soluções, sem considerar o tempo de execução, os resultados indicam que taxas mais elevadas, especialmente 20%, apresentaram o melhor desempenho geral, principalmente ao levarmos em consideração a porcentagem de soluções ótimas.

A taxa de elitismo de 20% obteve maior proporção de execuções que resultaram em soluções ótimas (90%), embora tenha apresentado a média do número de movimentos um pouco maior, juntamente com a média de gerações que encontram a solução do problema.

Por outro lado, as taxas mais baixas, como a de 1% , apesar de apresentar uma convergência mais rápida, mostrou resultados inferiores em termos de qualidade da solução, com menores taxas de sucesso (70%).

Portanto, a taxa de elitismo com 20% mostrou ser a opção mais indicada, não apenas por obter a solução ótima com maior frequência comparado a outras taxas, mas também por ter apresentado uma boa média de número de movimentos e número de gerações. Desta forma, para os testes com a taxa de elitismo o critério principal usado para definir a melhor configuração foi a porcentagem de soluções ótimas encontradas.

Segue a tabela comparativa (Tabela 11) dos resultados dos experimentos realizados para a taxa de elitismo.

Tabela 11 – Resumo comparativo entre as taxas de elitismo,

Taxa	% Soluções ótimas	n° Movimentos (média)	Geração (média)
1%	70%	27	3
5%	70%	26	136
15%	80%	28	44
20%	90%	29	63

4.3 Experimentos com a taxa de *crossover*

Nesta seção, foram realizados os testes relacionados a taxa de *crossover* utilizada na implementação do Algoritmo Genético desenvolvido neste trabalho. Nas Tabelas 12, 13, 14 e 15, são expostos os dados obtidos dos testes realizados para a análise do comportamento do AG para diferentes taxas de cruzamento. Nesta etapa, foram utilizados como parâmetros: população (200); número máximo de gerações (2000); taxa de mutação (15%); taxa de elitismo (20%).

Tabela 12 – Taxa de *crossover* = 100%.

Execução	Geração	n° Movimentos
1	3	38
2	4	41
3	111	18
4	6	35
5	2	17
6	750	16
7	1	40
8	1	28
9	1797	24
10	250	19
11	1	29
12	2001	20
13	1	18
14	2001	15
15	3	34
16	1	16
17	4	31
18	1	15
19	9	28
20	2001	20
Média	173	25
Desvio Padrão	802	9

Na configuração com taxa de *crossover* igual a 100% (Tabela 12), o Algoritmo Genético obteve a solução ótima em 17 das 20 execuções. A média de gerações necessárias para encontrar a solução foi 173, com um desvio padrão populacional elevado de 802, indicando grande variação no desempenho. Em relação ao número médio de movimentos das soluções, foi obtido um valor igual a 25 movimentos, com desvio de 9.

Tabela 13 – Taxa de *crossover* = 95%.

Execução	Geração	n° Movimentos
1	3	21
2	1	27
3	6	35
4	5	41
5	2	32
6	2001	15
7	4	39
8	1	29
9	2001	22
10	1	33
11	1	15
12	2	36
13	1	19
14	2001	31
15	2001	19
16	2001	26
17	1	15
18	7	22
19	2001	28
20	1	25
Média	3	27
Desvio Padrão	2	8

Para a taxa de *crossover* igual a 95% (Tabela 13), o Algoritmo Genético encontrou a solução do problema em 14 das 20 execuções, pior que o resultado anterior. O número médio de gerações necessárias para encontrar a solução foi 3, com um desvio padrão populacional de 2. O número médio de movimentos das soluções foi igual a 27, com desvio de 8.

Tabela 14 – Taxa de *crossover* = 90%.

Execução	Geração	n° Movimentos
1	2001	15
2	2	37
3	1	20
4	2001	38
5	7	40
6	1	24
7	3	36
8	5	24
9	2	26
10	2001	16
11	1448	17
12	4	38
13	1	31
14	3	24
15	5	26
16	5	19
17	3	21
18	2001	16
19	2001	20
20	2001	18
Média	106	25
Desvio Padrão	372	8

Na configuração com taxa de *crossover* igual a 90% (Tabela 14), o Algoritmo Genético encontrou a solução em 14 das 20 execuções, semelhante ao experimento com 95%. O número médio de gerações necessárias para encontrar a solução foi de 106, com um desvio padrão populacional de 372. O número médio de movimentos das soluções obtidas foi igual a 25, com desvio de 8.

Tabela 15 – Taxa de *crossover* = 80%.

Execução	Geração	n° Movimentos
1	11	28
2	5	30
3	1	20
4	2	18
5	3	33
6	1293	43
7	1	26
8	47	38
9	1	37
10	4	18
11	2	31
12	1	28
13	1	33
14	43	43
15	18	27
16	2001	27
17	1	40
18	2	27
19	1	30
20	2001	16
Média	80	30
Desvio Padrão	295	8

Com a taxa de *crossover* ajustada para 80% (Tabela 15), o Algoritmo Genético obteve a solução do problema em 18 das 20 execuções, conseguindo superar todos os experimentos anteriores neste quesito. O número médio de gerações necessárias foi igual a 80, com um desvio padrão populacional de 295. O número médio de movimentos das soluções encontradas foi igual a 30.

4.3.1 Análise dos resultados

Os experimentos realizados com diferentes taxas de *crossover* demonstraram a influência deste parâmetro no desempenho do Algoritmo Genético para o problema do 8-puzzle.

Dentre os cenários testados, a taxa de a 80% apresentou o melhor desempenho geral. Essa configuração foi capaz de encontrar a solução ótima em 18 das 20 execuções, representando a melhor performance entre todas as opções avaliadas.

Embora o número médio de gerações (80) e o número médio de movimentos das soluções (30 movimentos) tenham sido ligeiramente superiores aos de outras configurações, esses valores se mostraram aceitáveis ao considerar à alta taxa de sucesso em encontrar a

solução ótima e à boa qualidade das soluções. Também como nos experimentos anteriores, a porcentagem das soluções ótimas encontradas foi o determinante para indicar a melhor configuração para este problema.

Segue a tabela comparativa dos resultados dos experimentos realizados para a taxa de *crossover* (Tabela 16).

Tabela 16 – Resumo comparativo entre as taxas de *crossover*.

Taxa	% Soluções ótimas	n° Movimentos (média)	Geração (média)
80%	90%	30	80
90%	70%	25	106
95%	70%	27	3
100%	85%	25	173

4.4 Comparação com trabalho relacionado

Nesta seção são apresentados testes comparativos entre o Algoritmo Genético (AG) desenvolvido neste Trabalho de Conclusão de Curso e os resultados obtidos por [Yang et al. \(2023\)](#). A descrição dos dois AGs está na Tabela 17.

Tabela 17 – Comparação entre os AGs do trabalho de referência e do trabalho desenvolvido.

Característica	Trabalho de Referência	Trabalho desenvolvido
Seleção	Seleção por Roleta. Não há elitismo.	Elitismo + Seleção por Roleta. Melhor preservação de indivíduos.
Mutação	Mutação de adição ou modificação. Não evita movimentos redundantes.	Mutação de adição ou modificação. Evita movimentos que se anulam (ex: direita seguido de esquerda).
<i>Crossover</i>	<i>Crossover</i> uniforme (baseado em probabilidade).	<i>Crossover</i> de um ponto de corte (determinístico). Prioriza a herança do melhor pai.
Inicialização	População inicial com genes vazios.	População inicial com genes aleatórios de tamanho entre 15 e 40.
Custo/ <i>Fitness</i>	Utiliza a distância Euclidiana.	Utiliza a distância de Manhattan (<i>custoManhattan</i>).

Os experimentos foram realizados em vinte execuções para cada configuração da taxa de mutação (10%, 50% e 90%), mantendo os mesmos parâmetros utilizados no estudo de referência: população de 100 indivíduos, número máximo de 1000 gerações, taxa de *crossover* de 50% e elitismo de 0%.

Note que embora os dois AGs utilizem funções de aptidão diferentes, ambas buscam minimizar a distância de uma solução candidata para a solução alvo. Além disto, ambos resolvem o mesmo problema de otimização, tentando minimizar o número de gerações necessárias para encontrar a solução ótima. Neste trabalho, utilizou-se a distância de Manhattan aplicada ao problema 8-puzzle, pois é mais apropriada do que a distância Euclidiana, uma vez que 8-puzzle é um jogo de tabuleiro com peças dispostas em uma grade 3x3, onde cada peça pode ser movida apenas para cima, baixo, esquerda ou direita, nunca na diagonal. Isso reflete exatamente o conceito da distância de Manhattan (IORDAN, 2016). Outro ponto importante é a inicialização da população, em que neste trabalho foi implementada a população inicial com genes aleatórios de tamanho entre 15 e 40, diferente do trabalho de Yang et al. (2023). Este método foi analisado por meio de experimentos e concluiu-se que esta inicialização obteve resultados melhores que a de Yang et al. (2023).

De forma geral, considerando a média condicional das gerações quando o algoritmo encontrou as soluções ótimas (ou seja, a média sobre execuções bem-sucedidas), o AG desenvolvido neste trabalho apresentou desempenho superior ao de Yang, alcançando a solução ótima em um número menor de gerações, especialmente para as taxas de mutação mais altas.

Na Tabela 18 estão os resultados de Yang et al. (2023). Neste caso, a melhor configuração foi a taxa de mutação de 90%, com média condicional de 51,75 gerações, seguida pela taxa de 50% (218,65 gerações). A taxa de 10% foi a menos eficiente, apresentando média de 338,05 gerações. A Tabela 19 mostra os resultados do AG desenvolvido neste trabalho. Assim como no estudo de referência, a taxa de mutação de 90% também apresentou o melhor desempenho, mas com média significativamente menor (19,13 gerações). A taxa de 50% obteve resultado próximo (20,73 gerações), enquanto a taxa de 10% foi a menos eficiente (147,86 gerações). Isto demonstra que este algoritmo tem a capacidade de convergir mais rapidamente para a solução ótima, quando ele é bem sucedido. No entanto, ele apresenta execuções em que a solução ótima não foi encontrada, não sendo tão confiável quanto o de Yang.

Tabela 18 – Trabalho de referência - Taxa de mutação.

Execução	10%	50%	90%
1	544	1	2
2	680	7	54
3	369	97	21
4	235	564	23
5	179	244	69
6	4	268	77
7	9	6	39
8	149	127	145
9	567	329	143
10	9	342	41
11	431	5	63
12	999	913	8
13	196	178	1
14	131	237	30
15	11	154	7
16	761	2	48
17	173	4	105
18	407	74	38
19	899	272	111
20	8	549	10
Média	338,05	218,65	51,75
Desvio Padrão	305	231,3	44

Tabela 19 – Experimento com o AG desenvolvido - Taxa de mutação.

Execução	10%	50%	90%
1	546	19	139
2	12	15	1001
3	41	3	4
4	3	7	2
5	196	34	1
6	1001	1001	1001
7	1	1001	25
8	23	5	4
9	3	1001	10
10	619	10	12
11	39	1	50
12	1001	6	6
13	1001	11	28
14	1	25	1
15	419	1	1001
16	148	1001	5
17	19	1001	1001
18	1001	6	5
19	1001	156	2
20	1001	12	12
Média	147,86	20,73	19,13
Desvio Padrão	210	37,2	33

4.4.1 Análise dos resultados

Ao comparar os resultados obtidos pelo AG desenvolvido neste trabalho com os apresentados por [Yang et al. \(2023\)](#), observa-se um padrão comum: taxas de mutação elevadas resultaram, em média, em menor número de gerações necessárias para alcançar a solução ótima. Como a taxa de cruzamento é de 50% nos dois AGs, a mutação neste caso teve uma importância significativamente maior do que nos experimentos anteriores, em que se tinha 100% de cruzamento; ou seja, para que houvesse maior diversidade foi necessário investir no aumento da taxa de mutação, ratificando a literatura da área.

Na investigação de [Yang et al. \(2023\)](#), a taxa de mutação de 90% apresentou o melhor desempenho (51,75 gerações, em média), seguida pela taxa de 50% (218,65 gerações, em média) e 10% (338,05 gerações, em média) com taxa de sucesso de 100% nas execuções. Isto significa que em todas as execuções este algoritmo encontrou a solução ótima, demonstrando uma alta confiabilidade na busca pelo resultado ótimo.

No AG desenvolvido neste trabalho, tanto a taxa de mutação de 90% (19,13 gerações, em média condicional) quanto a de 50% (20,73 gerações, em média condicional) alcançaram médias de iterações muito próximas e significativamente inferiores aos obtidos com 10% (147,86 gerações, em média condicional). Esta diferença de comportamento entre 50% e 90% sugere que a implementação proposta neste trabalho responde de forma eficiente a taxas intermediárias e altas, mantendo diversidade genética e evitando convergência

prematura. Isto se explica provavelmente pela mutação implementada neste trabalho, uma vez que ela evita movimentos que se anulam, agilizando o processo evolutivo.

Portanto, para definir o algoritmo mais indicado para um dado problema, é necessário identificar os critérios que devem ser considerados na análise: (i) a confiabilidade em encontrar soluções ótimas, ou (ii) a minimização do número de gerações necessárias para alcançar a solução ótima, tolerando algumas execuções sem sucesso. Deste modo, de acordo com o critério (i) o AG de [Yang et al. \(2023\)](#) teve melhor desempenho, e segundo o critério (ii), o AG desenvolvido neste trabalho foi mais adequado. Esta é a grande vantagem de se trabalhar com algoritmos genéticos, pois se pode avaliar não apenas uma única solução devolvida, mas todo o conjunto de possíveis soluções retornadas, analisando inclusive outros critérios importantes, como neste caso, o número de gerações requeridas para encontrar a solução ótima.

4.5 Considerações Finais

Os experimentos realizados com diferentes configurações dos parâmetros do Algoritmo Genético — taxa de elitismo, taxa de *crossover* e taxa de mutação — permitiram avaliar com clareza o impacto de cada um deles na resolução do problema do 8-puzzle. As análises levaram em conta principalmente a qualidade das soluções geradas, a taxa de sucesso na obtenção do estado objetivo e a consistência entre as execuções.

A taxa de elitismo de 20% destacou-se como a configuração mais eficaz, alcançando 90% de taxa de sucesso e soluções curtas, com média de 29 movimentos. Em relação à taxa de *crossover*, observou-se que a taxa de 80% proporcionou os melhores resultados gerais, com alta taxa de sucesso (90%), além de ter obtido uma boa média para o número de movimentos das soluções. A análise da taxa de mutação mostrou que a configuração de 15% foi a mais adequada, apresentando alta taxa de sucesso (85%) — empatando com a taxa de 1% —, além de gerar soluções mais curtas (25 movimentos) e com baixa variabilidade.

Com base nestes resultados, conclui-se que a combinação dos parâmetros elitismo de 20%, *crossover* de 80% e mutação de 15% proporciona um equilíbrio eficaz entre qualidade, estabilidade e eficiência na busca por soluções para o 8-puzzle. Os experimentos realizados fornecem uma base sólida para a parametrização inicial do algoritmo e reforçam a importância da calibração cuidadosa dos parâmetros em aplicações que exigem soluções de alta qualidade e comportamento consistente.

Por fim, ao comparar o AG proposto com a abordagem de [Yang et al. \(2023\)](#), observou-se um desempenho promissor, especialmente quando considerada a média condicional dos melhores valores de *fitness*, que apresentou menores médias e desvios padrão. No entanto, ao se analisar exclusivamente a taxa de sucesso nas 20 execuções, o AG de

[Yang et al. \(2023\)](#) demonstrou desempenho superior. Essa análise reforça a ideia de que os algoritmos genéticos possuem a vantagem de poder considerar múltiplos critérios na escolha da abordagem mais apropriada.

5 Conclusão

Este Trabalho de Conclusão de Curso teve como objetivo principal propor a aplicação de um Algoritmo Genético para a resolução do problema do quebra-cabeça de 8 peças. Para tal, a implementação e a experimentação com diferentes configurações dos parâmetros genéticos — mutação, cruzamento e elitismo — foram etapas fundamentais para avaliar o desempenho do algoritmo em função das variações destes parâmetros. Como objetivo específico, buscou-se realizar, por meio de experimentos, uma análise comparativa entre os operadores e configurações adotadas no algoritmo implementado com o que há descrito na literatura, mais especificamente com o trabalho de Yang (YANG et al., 2023).

A partir da etapa de experimentação realizada nesta pesquisa, foi possível concluir a relevância do ajuste dos parâmetros utilizados pelo algoritmo para a obtenção de bons resultados. Observou-se que, com a escolha adequada dos operadores de mutação, *crossover* e elitismo, o algoritmo apresentou um bom desempenho, sendo capaz de encontrar a solução ótima do problema em um número reduzido de iterações e com alta frequência, superando os resultados obtido por (YANG et al., 2023) se for considerada a média condicional das gerações. Se for considerada a taxa de sucesso na busca por soluções ótimas, o AG de (YANG et al., 2023) performa melhor encontrando a solução ótima em 100% das execuções, enquanto o AG desenvolvido neste trabalho tem uma taxa de sucesso de 80%. Tal análise evidencia a flexibilidade dos algoritmos genéticos em incorporar múltiplos critérios na escolha da estratégia mais eficiente.

Para trabalhos futuros, sugere-se investigar o impacto de diferentes tipos de operadores de *crossover* na qualidade das soluções e na convergência do algoritmo, tais como o *uniform crossover*, *order crossover*, como também mutações baseadas em heurísticas, além de explorar outros métodos de inicialização da população para este problema, com o objetivo de melhorar a taxa de sucesso deste AG. Ademais, pretende-se estender o algoritmo para variantes mais desafiadoras do problema, como o 15-puzzle ou outros problemas de busca, avaliando o desempenho e escalabilidade da solução.

Referências

- AMABIS, J. M.; MARTHO, G. R. **Curso básico de biologia**. São Paulo: Editora Moderna Ltda., 1985. v. 3. Citado na página 16.
- BEZULJ, M. P.; JOVICIC, N. S. Genetic algorithm-based solver for jigsaw puzzles - analysis and improvement. In: **2018 26th Telecommunications Forum (TELFOR)**. Belgrade, Serbia: [s.n.], 2018. Citado na página 23.
- BUKHORI, I.; FELIX, J.; ALI, S. Using genetic algorithm to solve puzzle games: A review. **Journal of Computer Networks, Architecture and High Performance Computing**, v. 6, n. 1, 2024. Disponível em: <<https://doi.org/10.47709/cnahpc.v6i1.3348>>. Citado na página 23.
- COELLO, C. A. C.; VELDHUIZEN, D. A. V.; LAMONT, G. B. **Evolutionary Algorithms for Solving Multi-Objective Problems**. New York, NY: Kluwer Academic, 2002. (Genetic Algorithms and Evolutionary Computation). Citado na página 16.
- DARWIN, C. **On the Origin of Species**. London: John Murray, 1859. 502 p. Citado 2 vezes nas páginas 10 e 13.
- DEB, K. **Multi-objective Optimization Using Evolutionary Algorithms**. New York, NY: John Wiley & Sons, 2001. (Wiley-Interscience Series in Systems and Optimization). Citado na página 15.
- DEZA, M. M.; DEZA, E. **Encyclopedia of Distances**. [S.l.]: Springer, 2009. Citado na página 26.
- EIBEN, A. E.; SMITH, J. E. **Introduction to Evolutionary Computing**. Berlin, Heidelberg: Springer, 2003. Citado na página 10.
- FOGEL, D. B. An introduction to simulated evolution. **IEEE Transactions on Neural Networks**, v. 5, n. 1, 1994. Citado na página 19.
- FONTOURA, A. J.; PACCE, T. A.; SCHREIBER, J. N. C. Grh – genetic rush hour: Uma heurística genética na solução de problemas pspace-completo. In: **XVIII Congreso Argentino de Ciencias de la Computación**. [s.n.], 2012. Disponível em: <<http://sedici.unlp.edu.ar/handle/10915/23589>>. Disponível em: <<http://sedici.unlp.edu.ar/handle/10915/23589>>. Citado na página 22.
- GABRIEL, P. H. R.; DELBEM, A. C. B. **Fundamentos de Algoritmos Evolutivos**. [S.l.]: Universidade de São Paulo, 2008. Citado 7 vezes nas páginas 10, 13, 14, 15, 16, 17 e 19.
- GOLDBERG, D. E. **Genetic Algorithms in Search, Optimization, and Machine Learning**. New York: Addison-Wesley, 1989. Citado 3 vezes nas páginas 13, 16 e 17.
- HOLLAND, J. H. **Adaptation in Natural and Artificial Systems**. Michigan: The University of Michigan Press, 1975. Citado 2 vezes nas páginas 10 e 13.

- IGWE, K.; PILLAY, N.; RAE, C. Solving the 8-puzzle problem using genetic programming. 2013. Citado na página 21.
- IORDAN, A.-E. A comparative study of three heuristic functions used to solve the 8-puzzle. **Journal of Advances in Mathematics and Computer Science**, v. 16, n. 1, p. 1–18, 2016. Disponível em: <<https://journaljamcs.com/index.php/JAMCS/article/view/481>>. Citado na página 46.
- JONG, K. A. D. **Evolutionary Computation: A Unified Approach**. Cambridge, MA: MIT Press, 2006. Citado na página 13.
- JUNIOR, N. F.; GUIMARAES, F. G. Problema 8-puzzle: Análise da solução usando algoritmos genéticos e backtracking. 2011. Citado na página 22.
- LEONARD, R. J. From parlor games to social science: Von neumann, morgenstern, and the creation of game theory 1928-1944. **Journal of Economic Literature**, v. 33, 1995. Citado na página 20.
- LEYTON-BROWN, K.; SHOHAM, Y. **Essentials of Game Theory: A Concise, Multidisciplinary Introduction**. [S.l.]: Morgan & Claypool, 2008. Citado na página 20.
- MAGALHAES, S. Tenreiro de; Sá, V. Establishment of automatization as a requirement for time management input modules in project management information systems for academic activities – a game theory approach. **Procedia Computer Science**, v. 64, p. 1157–1162, 2015. Citado na página 20.
- MICHALEWICZ, Z. **Genetic Algorithms + Data Structures = Evolution Programs**. 3. ed. Berlin: Springer, 1996. Citado 3 vezes nas páginas 14, 15 e 19.
- MILLER, B. L.; GOLDBERG, D. E. Genetic algorithms, tournament selection, and the effects of noise. In: **Complex Systems**. University of Illinois at Urbana-Champaign, USA: University of Illinois, 1995. Citado na página 19.
- NEUMANN, J. V.; MORGENSTERN, O. **The Theory of Games and Economic Behavior**. 1. ed. Princeton, NJ: Princeton University Press, 1944. ISBN 9780691130613. Citado na página 20.
- QIAN, T. **Using Genetic Algorithm to Solve Sliding Tile Puzzles**. 1995. Cognitive Science Program, Oswego, USA. Citado na página 21.
- SANTOS, M. M. d.; FAGUNDES, M. F. C.; FERREIRA, N. B.; NESTERIUK, S. Aplicação simultânea das teorias de jogos de soma zero e de soma não-zero no design de um jogo de tabuleiro digital. In: **Curso de Design de Games**. Universidade Anhembi Morumbi, São Paulo, Brasil: [s.n.], 2018. Citado na página 21.
- SARTINI, B. A.; GARBUGIO, G.; BORTOLOSSI, H. J.; SANTOS, P. A.; BARRETO, L. S. **Uma Introdução à Teoria dos Jogos**. São Paulo, SP: IME-USP, 2004. Disponível em: <<https://www.ime.usp.br/~rvicente/IntroTeoriaDosJogos.pdf>>. Citado na página 20.
- SHABAN, R. Z.; ALKALLAK, I. N.; SULAIMAN, M. M. Genetic algorithm to solve sliding tile 8-puzzle problem. **Journal of Education and Science**, v. 23, n. 3, 2010. Citado 4 vezes nas páginas 11, 21, 22 e 23.

SMITH, J. M. **Evolution and the Theory of Games**. Cambridge: Cambridge University Press, 1982. Citado na página 20.

SUH, J. Y.; LEE, C. D. Extending distributed genetic algorithms to problem solving: The case of the sliding block puzzle. 1989. Publicado em 1989. Citado na página 22.

YANG, C. M.; PEK, V.; LING, S. H.; WEI, T. C.; SALAM, Z. A. A. Solver of 8-puzzle with genetic algorithm. **Journal of Applied Technology and Innovation**, v. 7, n. 1, 2023. Publicado em 2023. Citado 9 vezes nas páginas 11, 23, 25, 45, 46, 47, 48, 49 e 50.

ZUBEN, F. J. V. Computação evolutiva: Uma abordagem pragmática. In: **Anais da I Jornada de Estudos em Computação de Piracicaba e Região (1a. JECOMP)**. Piracicaba, SP: [s.n.], 2000. Citado 2 vezes nas páginas 14 e 19.